

CityTech TTP Summer Bootcamp

Collaboration and Developer Practices

2026-06-04

Agenda

1. Review
2. Follow Up Notes
3. GitHub Issues and Feedback
4. Real-World Repos
5. Independent Learning
6. Efficiency
7. Stand-ups
8. Writing Good Issues
9. Participatory Learning

Review

What are the most important things you remember from yesterday?

Follow Up Notes - Port Blocking

Why might **CityTech-WiFi** block your SSH connections?

- Campus and guest networks often allow only common web ports (80/443) and block the rest - including SSH's **port 22**.
- SSH can open encrypted tunnels that bypass content filters and monitoring, so it's commonly blocked by policy.
- Blocking port 22 shrinks the attack surface and stops compromised devices from reaching outside SSH servers.
- It's a network-policy decision - not a problem with your machine or your keys.

`/home` vs. `~`

Your **home directory** is your personal folder - where your files, settings, and dotfiles live. Every user has their own.

- `~`: a shortcut that always expands to **your** home directory
 - `cd ~` → go home · `~/projects` → your projects folder
- `/home`: on Linux/WSL, the folder that holds **everyone's** home directories (`/home/jon`, `/home/maria`)
- So your home is `/home/<username>` - `/home` itself is just the parent
- On **macOS**, homes live under `/Users` instead (so `~` = `/Users/yourname`)

What Are GitHub Issues?

An **issue** is a tracked item on a repository - a bug report, task, question, or note - with its own title, description, comments, and open/closed status.

Why teams use them:

- **Track work:** bugs, features, and tasks in one place
- **Discuss:** comment threads tied to a specific piece of work
- **Organize:** assignees, labels, and milestones
- **Link code:** connect commits and pull requests to the issue they resolve

Getting Feedback

- Usually, the instructional team will leave **issues** on your GitHub repo
- This mimics what you'd experience in the workplace
- Forked repos don't automatically have issues enabled

Enabling Issues on a Repo

1. Open your repository on **GitHub**
2. Click the **Settings** tab
3. Scroll to the **Features** section (under **General**)
4. Check the **Issues** box

Guide: <https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/enabling-features-for-your-repository/disabling-issues>

Examining Real-World Repos

See how real projects are organized - READMEs, issues, pull requests, structure:

- **Our World in Data - Grapher**: data and charts on global problems
<https://github.com/owid/owid-grapher>
- **Ushahidi Platform**: crowdsourced crisis mapping and citizen reporting
<https://github.com/ushahidi/platform>
- **OpenStreetMap - iD editor**: the in-browser editor for the free world map
<https://github.com/openstreetmap/iD>

Related Terms and Platforms

GitHub Issues is one of many **issue trackers** - same idea, different names:

- **Ticket**: the general term for a tracked unit of work (a GitHub issue *is* a ticket)
- **Jira**: Atlassian's widely used tracker, very common in industry
- Others you'll meet: **Linear, Trello, Asana, Azure Boards**
- **Backlog**: tickets not yet started · **Sprint** - a time-boxed batch of work

Independent Learning - Why

- Tech changes fast (especially AI) - you can't wait to be taught
- When new tech drops, you have two options:
 - i. Struggle and learn it yourself
 - ii. Wait for someone else to learn it, then teach you (if that's even possible)
- Option 1 keeps you **current and knowledgeable**
- And when **job hunting**, it puts you at the front - option 2 leaves you behind

Independent Learning - What It Means

- It doesn't mean learning *only* on your own
- It means **you** are in control - and proactive
- Sometimes that's asking others for help - but you drive it

Efficiency - Why It Matters

- In the real world, **time** is one of your most limited resources
- Deadlines; other teams waiting on your code
- Code must stay reliable - but generally faster is better
- Faster work = more free time to enjoy life; **burnout is real**

Efficiency - Tools and Caveats

- You'll pick up tips and tricks: **scripts and one-liners**
- AI can produce code fast - but it's a **black box**; the quality still needs verifying

What Is a Stand-up?

A short **daily** team meeting (often ~15 min) to keep everyone in sync - a staple of Agile/Scrum.

Each person answers three things:

1. What I did **yesterday**
2. What I'm doing **today**
3. Any **blockers**

Blocker - anything stopping your progress: a bug, missing access, unclear requirements, or waiting on someone else.

Stand-ups - As a Junior Dev

- Keep it **short and specific**: yesterday → today → blockers
- Having blockers is normal and expected - **raise them early**
- Don't solve a blocker live - note it and "**take it offline**" with the right person afterward
- If you're stuck, **say so**; staying silent hides problems and slows the team
- Listen for where others can unblock you - or where you can help them

Continue Play to Practice

1. <https://overthewire.org/wargames/bandit/>
2. <https://web.mit.edu/mprat/Public/web/Terminus/Web/main.html>
3. <https://github.com/veltman/clmystery>

Try to stay with the class - don't move on to the next game on your own.

Personal Projects

For building your GitHub profile and your employability, you want **as many projects as possible**.

- **Academic projects:** usually fine to include
- **Internship / job projects:** often *can't* be shared (intellectual property concerns)
- **Personal projects:** your own builds, always yours to show

The *topic* matters less than the skills - a weather app, a stock tracker, or a global-equality dashboard all work. **Build what interests you** to demonstrate what you can do.

Writing Good GitHub Issues

- **Descriptive title:** summarize the topic in one line
- **One topic per issue:** keep it focused
- **Give context:** what you're looking at and what you already understand
- **Be specific:** just like a good AI prompt, precise details get you better help
- **Identify** the file, line, or slide you're referring to
- **Format clearly:** use Markdown and put code in code blocks

Good Issues - Bugs vs. Questions

For a **bug report** - the goal is for maintainers to **reproduce it**:

- Steps to reproduce
- What you expected vs. what actually happened
- **System context** when relevant - operating system, version numbers, etc.

For a **question**:

- Your current understanding
- The specific point you're unsure about
- What you've already tried or read

Search first to avoid duplicates; stay concise and respectful.

Your GitHub Profile Matters

When you're job hunting, your GitHub profile **backs up your résumé**.

- **Your code** represents your knowledge, style, and thinking patterns
- **Your contributions** (commits, PRs, issues) show your interests and how you work
- **Real projects** beat an empty profile - it's a living, public portfolio
- Many people read the résumé first, then **verify it against your GitHub** - if the two don't match, that's a **red flag**

Participatory Learning

On our **main program repo**, **open or contribute to at least 3 issues** with questions about concepts from this week (the last 4 days).

- **Search existing issues first:** if a similar one already exists, contribute there instead of opening a duplicate
- **One issue per concept**
- Your question can be to:
 - **Clarify** something confusing
 - **Confirm** your current understanding is correct
 - **Explore** a concept further

Example Concepts and Questions

- **Staging area:** "Confirming: does `git add` snapshot the file *now*, so later edits need re-adding?"
- **Merge vs. rebase:** "When would I pick rebase over merge - and why does it rewrite commits?"
- **CORS:** "Does CORS block the request, or only my JavaScript from reading the response?"
- **SPAs:** "On a route change, what actually gets fetched if the page never fully reloads?"
- **~ vs. /home** : "Is `~` on macOS (`/Users/me`) the same idea as Linux's `/home/me` ?"

Thank You